Unit 3: Relational Model

Terminology:

1. Relation:

A relation is table with columns and rows.

2. attribute:

The columns in a relation are known as attributes.

3. Domain:

A domain or attribute domain is the set of allowable values for one or more attributes.

4. Tuples:

A tuple is a relation. They are also called the records.

5. Degree:

The total numbers of columns or attributes is the relation.

6. Cardinality:

Total number of rows present in the Table.

7. Relation Schema: A relation schema represent the names of the relation with its attributes.

Table/Relation

Attribute/Column

Students

Primary Key: Roll_No

Roll_No	Name	Phone_No
1	Ajay	9898373232
2	Raj	9874444211
3	Vijay	8923423411
4	Aman	8886462644

Tuple/Row

Cardinality

Domain
(Eg: Interger 0-9)

Tuple variable

Degree (no. of columns)=3

Relation Schema: Students (Roll_No, Name, Phone_No)

Notes:

1. Procedural Query Language:

- In procedural query language, user instructs the system to perform a series of operations to produce the desired result.
- User tells what data to be retrieved from database and how to retrieve it.

2. Non –procedural(or Declarative) Query Language:

- In non-procedural query language, user instructs the system to produce the desired result without telling the step by step process.
- User tells what data to be retrieved from database but doesn’t tell how to retrieve it.

Structure of relational databases

A relational database consists of a collection of relations (tables). Each relation has rows (tuples) and columns (attributes). Relationships between tables are established using Primary Keys and Foreign Keys.

Structure of a Relational Database

1. Relation (Table)
2. Attribute (Column)

Relational Algebra:

- Relational Algebra is a procedural query language used to perform operations on relations (tables). It takes one or more relations as input and produces a relation as output.
- It provides the theoretical foundation for relational databases and SQL.
- Being a procedural language, it specifies how to retrieve the required data by applying a sequence of operations.
- Relational Algebra operates on entire relations (tables) rather than processing one tuple (row) at a time. Therefore, there is no need to use loops to iterate through individual rows.
- It uses a set of operations such as Selection (σ), Projection (π), Union (∪), Set Difference (−), Cartesian Product (×), Rename (ρ), and Join (⋈) to manipulate and retrieve data.

Example:

- Relational algebra works on the entire table (relation) at once, so there is no need to use loops to process each row (tuple) individually.
- We only specify the table (relation) and the required operation, and relational algebra returns the required data in a single operation.

We can use Relational Algebra to fetch data from this Table(relation)

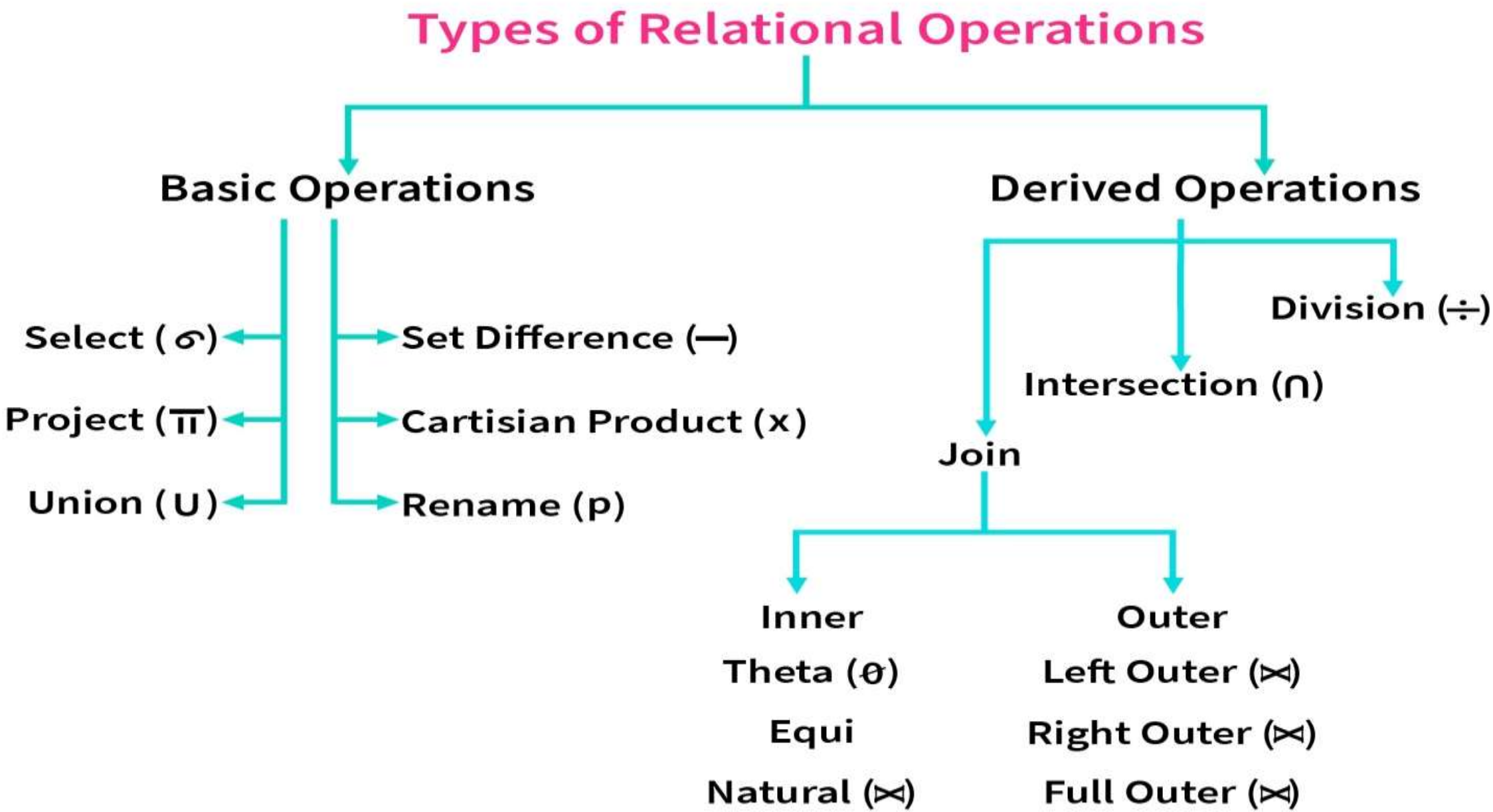
ID	Name	Age
1	Akon	17
2	Bkon	19
3	Ckon	15
4	Dkon	13

Select Name students with age less than 17

Output

Name
Ckon
Dkon

The output for query is also in form of a table(relation), with results in different columns


Types of Relational Operations
Basic Operations
Derived Operations
Select (σ)
Project (π)
Union (U)
Set Difference (—)
Cartisian Product (x)
Rename (p)
Division (÷)
Intersection (∩)
Join
Inner
Theta (θ)
Equi
Natural (⋈)
Outer
Left Outer (⋈)
Right Outer (⋈)
Full Outer (⋈)

SELECT(σ):

- "The SELECT command is **applied to a single table** and takes rows that meet a specific condition copying them into a new table."
- Informal general form:
  - **SELECT tableName WHERE condition [GIVING newTableName]**

  Symbolic form:

  $\sigma_{EmpDept = 10}(EMPLOYEE)$

  Meaning:

  This operation selects all employees from the **EMPLOYEE** table whose **EmpDept** is **10**.

Example:

Table: CUSTOMER

Customer_Id	Customer_Name	Customer_City
C10100	Steve	Agra
C10111	Raghu	Agra
C10115	Chaitanya	Noida
C10117	Ajeet	Delhi
C10118	Carl	Delhi

Query:

```
σCustomer_City = "Agra" (CUSTOMER)
```

Output:

Customer_Id	Customer_Name	Customer_City
C10100	Steve	Agra
C10111	Raghu	Agra

PROJECT(∏):

- PROJECT Operation is **denoted by ∏(pi).**
- It is used to **select specific columns (attributes)** from a relation and **discard the remaining columns**.
- PROJECT creates a **vertical partitioning.**
- The output contains **only the specified attributes**. If duplicate rows are produced, **PROJECT removes duplicate tuples** (since a relation cannot contain duplicate tuples).

Example: To list each employee's first and last name and salary, the following is used:

∏FNAME, LNAME,SALARY(EMPLOYEE)

The general form of the *project* operation is:

$\pi_{<\text{attribute list}>}(R)$

- π (pi) is the symbol used to represent the *project* operation
  <attribute list> is the desired list of attributes from relation R.
- The **project operation *removes any duplicate tuples.***
- This is because the result of the *project* operation must be a *set of tuples*

Project Operator (∏) Example:

In this example, we have a table CUSTOMER with three columns, we want to fetch only two columns of the table, which we can do with the help of Project Operator ∏.

Table: CUSTOMER

Customer_Id	Customer_Name	Customer_City
C10100	Steve	Ktm
C10111	Raghu	Ktm
C10115	Chaitanya	dhn
C10117	Ajeet	mnr
C10118	Carl	mnr

Query:

$\prod$ Customer_Name, Customer_City (CUSTOMER)

Output:

Customer_Name	Customer_City
Steve	Ktm
Raghu	Ktm
Chaitanya	dhn
Ajeet	mnr
Carl	mnr

Union Operator (∪):

- The **Union Operator (∪)** is a binary operator in relational algebra that combines all distinct tuples (rows) from two relations (tables) into a single relation. For the union operation to be valid, the two relations must be **union-compatible**.
- **Key Points:**

1.Union-Compatible Relations:

1. The two relations must have the **same number of attributes (columns)**.
2. The corresponding attributes must have the **same domain** (i.e., the same data type and meaning).

2.Distinct Tuples:

1. The union operation removes duplicate tuples, ensuring that the result contains only unique rows.

Syntax of Union Operator (∪)

```
table_name1 ∪ table_name2
```

Union Operator (∪) Example

Table 1: COURSE

Course_Id	Student_Name	Student_Id
C101	Aditya	S901
C104	Aditya	S901
C106	Steve	S911
C109	Paul	S921
C115	Lucy	S931

Table 2: STUDENT

Student_Id	Student_Name	Student_Age
S901	Aditya	19
S911	Steve	18
S921	Paul	19
S931	Lucy	17
S941	Carl	16
S951	Rick	18

Query:

∏ Student_Name (COURSE) ∪ ∏ Student_Name (STUDENT)

Output:

Student_Name
Aditya
Carl
Paul
Lucy
Rick
Steve

Note: As you can see there are no duplicate names present in the output even though we had few common names in both the tables, also in the COURSE table we had the duplicate name itself.

Set Difference (-):

Set Difference is denoted by – symbol. Lets say we have two relations R1 and R2 and we want to select all those tuples(rows) that are present in Relation R1
but **not** present in Relation R2, this can be done using Set difference R1 – R2.

Syntax of Set Difference (-)

```
table_name1 - table_name2
```

Set Difference (-) Example

Lets take the same tables COURSE and STUDENT that we have seen above.

Lets write a query to select those student names that are present in STUDENT table but not present in COURSE table.

```
∏ Student_Name (STUDENT) – ∏ Student_Name (COURSE)
```

Output:

```
Student_Name
------------
Carl
Rick
```

Cartesian product (X):

- Cartesian Product is denoted by X symbol. Lets say we have two relations R1 and R2 then the Cartesian product of these two relations (R1 X R2) would combine each tuple of first relation R1 with the each tuple of second relation R2.

- **Syntax of Cartesian product (X)**

  R1XR2

Cartesian product (X) Example

Table 1: R

Col_A	Col_B
AA	100
BB	200
CC	300

Table 2: S

Col_X	Col_Y
XX	99
YY	11
ZZ	101

Query:

Lets find the Cartesian product of table R and S.

R X S

Output:

Col_A	Col_B	Col_X	Col_Y
AA	100	XX	99
AA	100	YY	11
AA	100	ZZ	101
BB	200	XX	99
BB	200	YY	11
BB	200	ZZ	101
CC	300	XX	99
CC	300	YY	11
CC	300	ZZ	101

Note: The number of rows in the output will always be the cross product of number of rows in each table. In our example table 1 has 3 rows and table 2 has 3 rows so the output has 3×3 = 9 rows.

Rename (ρ i.e Rho):

- Rename (ρ) operation can be used to rename a relation or an attribute of a relation.

Rename (ρ) Syntax:

ρ(new_relation_name, old_relation_name)

Rename (ρ) Example:

- Lets say we have a table customer, we are fetching customer names and we are renaming the resulted relation to CUST_NAMES.

Table: CUSTOMER

Customer_Id	Customer_Name	Customer_City
C10100	Steve	Agra
C10111	Raghu	Agra
C10115	Chaitanya	Noida
C10117	Ajeet	Delhi
C10118	Carl	Delhi

Query:

```
ρ(CUST_NAMES, ∏(Customer_Name)(CUSTOMER))
```

Output:

```
CUST_NAMES
Steve
Raghu
Chaitanya
Ajeet
Carl
```

Derived Operations:

- Derived operations are operations that can be created using the basic operations of relational algebra. They don't add new power to the system, but they make it easier to write queries. The three main derived operations are:

1.Join
2.Intersection
3.Division

Join operation :

- **Join is an Additional which simplify the queries, but does not add any new power to the basic relational algebra.**
- Join is a combination of a Cartesian product followed by a selection process.

Join=Cartesian product + selection

- combines the relation R1 and R2 with respect to a condition. It is denoted by ⋈.

The different types of join operation are as follows –

- Inner join
- Theta join
- Equi join
- Natural join

Inner join

An Inner Join returns only the rows/tuples that have matching values in both relations according to the join condition.

Example:

STUDENT

SID	Name	Dept_ID
1	Ram	D01
2	Sita	D02
3	Hari	D03

DEPARTMENT

Dept_ID	Dept_Name
D01	Computer
D02	Science

After Inner Join on Dept_ID: Only matching records are included.

SID	Name	Dept_ID	Dept_Name
1	Ram	D01	Computer
2	Sita	D02	Science

Theta join

A Theta Join combines tuples from two relations when they satisfy a specified theta (θ) condition.

- It is a general form of join.
- It is used to join two or more relations based on a specified condition.
- The join condition is represented by the symbol θ.
- **It is uses all kinds of comparison operators like <,>,<=,>=.**
- **Notation: A ⋈θ B where θ is the condition for join.**

Example
Consider R1 table

RegNo	Branch	Section
1	CSE	A
2	ECE	B
3	CIVIL	A
4	IT	B
5	IT	A

Table R2

Name	RegNo
Ram	2
Sita	4

$R1 \bowtie_{\theta} R2$ with condition R1.regno > R2.regno

RegNo	Branch	Section	Name	Regno
3	CIVIL	A	Ram	2
4	IT	B	Ram	2
5	IT	A	Ram	2
5	IT	A	Sita	4

- In the join operation, we select those rows from the **Cartesian Product** where *R1.regno>R2.regno*.
- Join operation = select operation + cartesian product operation

2. Equi join:

An Equi Join is a special type of Theta Join that uses only the equality (=) operator as its join condition.

For example:

STUDENT ⋈ STUDENT.Dept_ID = DEPARTMENT.Dept_ID DEPARTMENT

Example of Equi Join

Suppose we have two relations: **STUDENT** and **DEPARTMENT**.

- **STUDENT Equi Join Condition**

STUDENT

Student_ID	Name	Dept_ID
101	Ram	D01
102	Sita	D02
103	Hari	D01
104	Gita	D03

DEPARTMENT

Dept_ID	Dept_Name
D01	Computer
D02	Science
D04	Management

- We join the two tables using the equality condition:

STUDENT.Dept_ID=DEPARTMENT.Dept_ID

So:

$$STUDENT \bowtie_{STUDENT.Dept_ID = DEPARTMENT.Dept_ID} DEPARTMENT$$

Result of Equi Join

Student_ID	Name	Dept_ID	Dept_ID	Dept_Name
101	Ram	D01	D01	Computer
102	Sita	D02	D02	Science
103	Hari	D01	D01	Computer

Why is Gita not included?Gita has Dept_ID = D03, but there is no D03 in the DEPARTMENT table, so there is no matching tuple.

Natural Join

- A Natural Join automatically joins two relations using their common attributes (columns) having the same name and compatible values.
- For example:

STUDENT

SID	Name	Dept_ID
1	Ram	D01
2	Sita	D02

DEPARTMENT

Dept_ID	Dept_Name
D01	Computer
D02	Science

- Natural Join: STUDENT ⋈ DEPARTMENTResult:

SID	Name	Dept_ID	Dept_Name
1	Ram	D01	Computer
2	Sita	D02	Science

- Notice that Dept_ID appears only once in the result.
- Key point: Natural Join uses common attributes automatically and removes duplicate join columns.

Outer Join:

- An Outer Join returns the matching tuples from two relations and also includes non-matching tuples from one or both relations. For missing values, NULL is used.
- It preserves tuples that do not have a matching tuple in the other relation.
- NULL generally represents a missing, unknown, or inapplicable value.

There are three kinds of outer joins:

1. **Left outer join**
2. **Right outer join**
3. **Full outer join**

1. Left outer join(⟕)

In left outer join, all the tuples from the left relation R1 are included in the resulting relation. The tuples of R1 which do not satisfy join condition will have values are null for attributes of R2.

In short:

All record from left table

Only matching records from right table

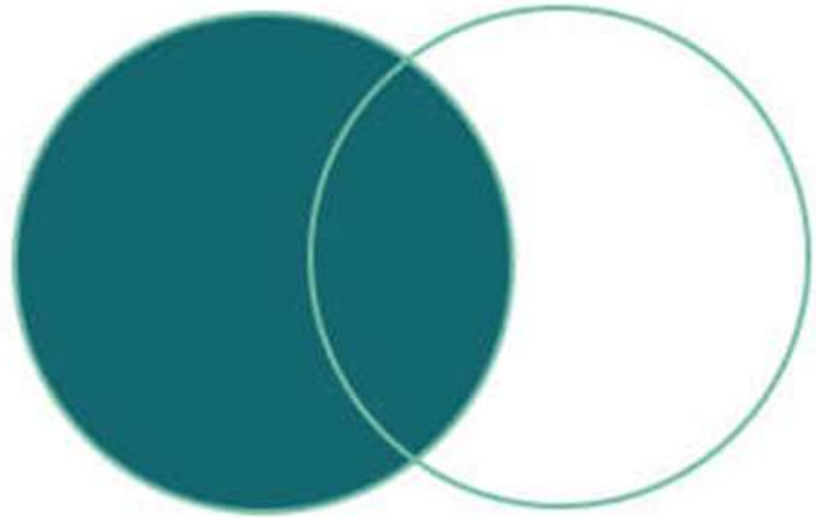
Symbol: ⟕

Notation: R1 ⟕ R2

LEFT OUTER JOIN in SQL

Left Table

Food	Price
Pizza	$8
Burger	$6
French Fries	$4



Right Table

Food	Quantity
Pizza	3
Burger	5
Cake	2

Food	Price	Quantity
Pizza	$8	3
Burger	$6	5
French Fries	$4	Null

2. Right outer join(⟖):

In right outer join, all the tuples from the right relation R2 are included in the resulting relation. The tuples of R2 which do not satisfy join condition will have values as NULL for attributes of R1.

In short:

All records from right table

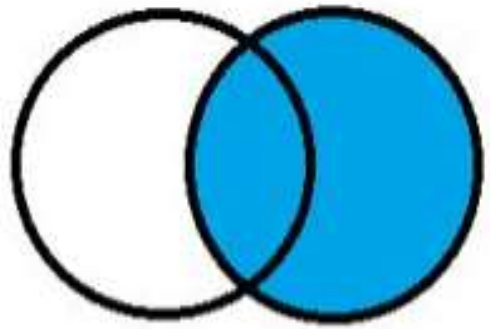
Only matching records from left table.

Symbol: ⟖

Notation: R1 ⟖ R2

Student ID	Name
1001	A
1002	B
1003	C
1004	D

Student ID	Department
1004	Mathematics
1005	Mathematics
1006	History
1007	Physics
1008	Computer Science



Student ID	Name	Department
1004	D	Mathematics
1005	NULL	Mathematics
1006	NULL	History
1007	NULL	Physics
1008	NULL	Computer Science

Full outer join:

In full outer join, all the tuples from both left relation R1 and right relation R2 are included in the resulting relation. The tuples of both relation R1 and R2 which do not satisfy join condition, their respective unmatched attributes are made NULL.

In short:

All records from all table.

Symbol: ⟗

Notation: R1 ⟗ R2

Example:

Student

S_id	Name	Class	Age	C_type
1	Andrew	5	25	A
2	Angel	10	30	A
3	Anamika	8	35	C

Course

C_type	C_name
A	Foundation C
B	C++

Student ⋈ Course

S_id	Name	Class	Age	C_type	C_name
1	Andrew	5	25	A	Foundation C
2	Angel	10	30	A	Foundation C
3	Anamika	8	35	C	-
-	-	-	-	B	C++

Intersection Operator (∩)

Intersection operator is **denoted by ∩ symbol** and it is used to **select common rows (tuples)** from two tables (relations).

Lets say we have two relations R1 and R2 both have same columns and we want to select all those tuples(rows) that are present in both the relations, then in that case we can apply intersection operation on these two relations R1 ∩ R2.

Note: Only those rows that are present in both the tables will appear in the result set.

Syntax of Intersection Operator (∩)

```
table_name1 ∩ table_name2
```

Intersection Operator (∩) Example

Lets take the same example that we have taken above.

Example:

- Consider two relations **Student1** and **Student2**:

ID	Name
1	Alex
2	Bob
3	Charlie

ID	Name
2	Bob
3	Charlie
4	David

Result of `Student1 ∩ Student2` **:**

ID	Name
2	Bob
3	Charlie

Table 1: COURSE

Course_Id	Student_Name	Student_Id
C101	Aditya	S901
C104	Aditya	S901
C106	Steve	S911
C109	Paul	S921
C115	Lucy	S931

Table 2: STUDENT

Student_Id	Student_Name	Student_Age
S901	Aditya	19
S911	Steve	18
S921	Paul	19
S931	Lucy	17
S941	Carl	16
S951	Rick	18

Query:

$\prod$ Student_Name (COURSE) $\cap$ $\prod$ Student_Name (STUDENT)

Output:

Student_Name

Aditya

Steve

Paul

Lucy

Division operation:

- Division is a **derived operator** (or additional operator).
- Suited to queries that include the keyword **“all” , or “every” like “at all” , “for all” or “in all”, “at every” or “in every”.**

Example:

- **Find students who have registered for every course.**
- Find employees who works on all project of company.

Notation: A ÷B OR A/B

- **Example:**
- **Given Relations:**
- **Relation R (Dividend)**

Student	Subject
A	Math
A	Science
B	Math
B	Science
B	English
C	Math

Relation S (Divisor)

Subject
Math
Science

Perform Division:

`R ÷ S` will return students who have taken **all subjects in S (Math & Science)**.

Result of R ÷ S (T)

Student
A
B

Here, Student A and B are in the result because they have both "Math" and "Science" in R. Student C is not included because they are missing "Science.“

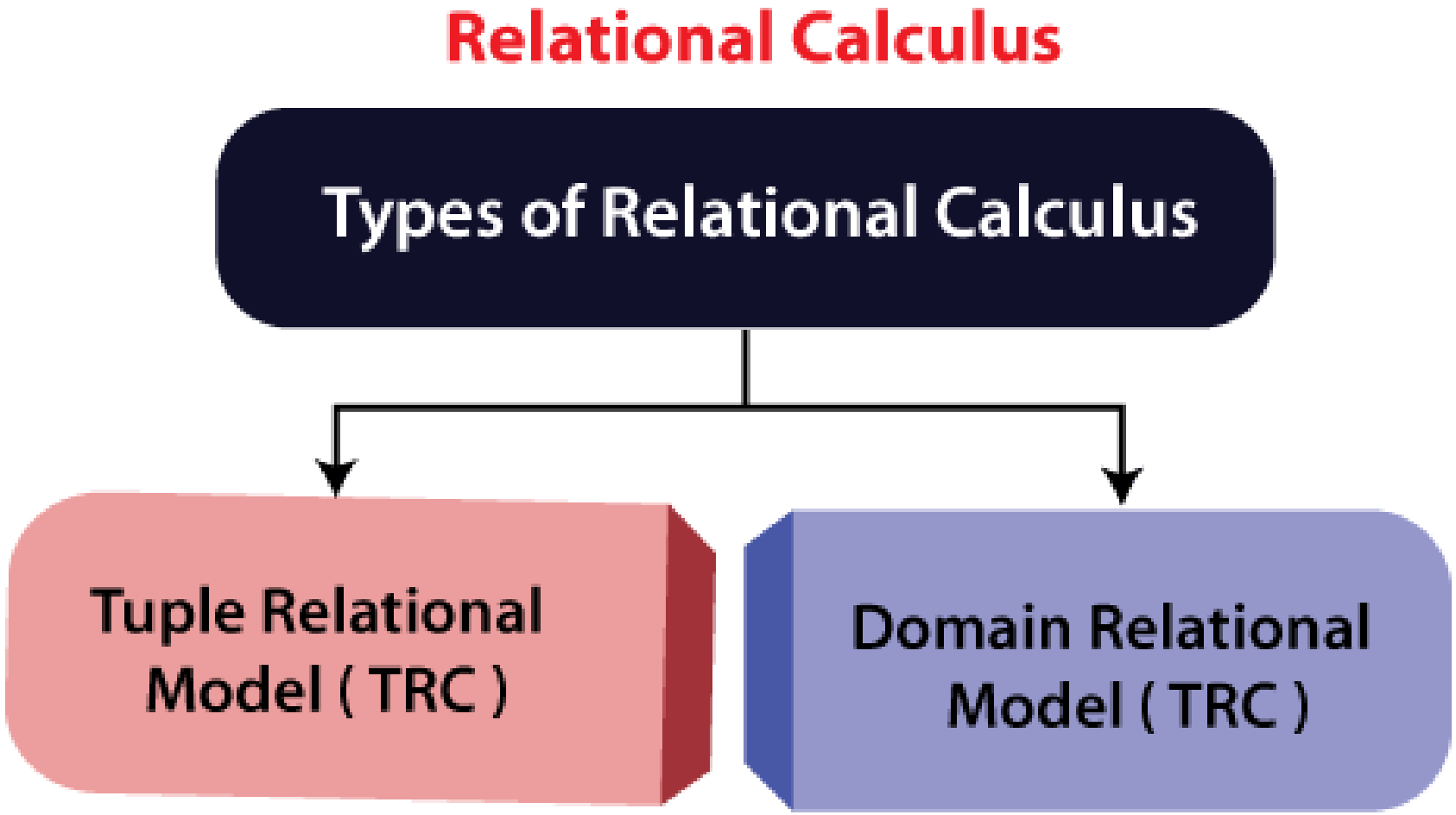
Relational Calculus:

- There is an **alternate way of formulating queries** known as Relational Calculus. Relational calculus is a **non-procedural query language**. In the non-procedural query language, the user is concerned with the details of how to obtain the end results. **The relational calculus tells what to do but never explains how to do.** Most commercial relational languages are based on aspects of relational calculus including SQL and QBE(Query By Example).

Common Notations Used in Relational Calculus:

- ∧ - represents AND.
- ∨ - represents OR.
- ¬ - represents NOT.
- ∈ - represents Belongs to.
- ∀ - represents that in a set of tuples, **all of them** satisfy the condition (Universal Quantifier).
- ∃ - represents that in a set of tuples, there is **at least one** occurrence that satisfies the condition (Universal Quantifier).

Type of relation calculus:



1. Tuple Relational Calculus (TRC):

- Tuple Relational Calculus (TRC) Tuple Relational Calculus (TRC) is a non-procedural (declarative) query language used in relational databases.
- It specifies what data is required, rather than how to retrieve the data.
- TRC uses tuple variables to represent tuples in a relation.

Notation:

A Query in the tuple relational calculus is expressed as following notation

{T | P (T)} or {T | Condition (T)}

Where

- **T** is the resulting tuples(final result).
- **P(T)** is the condition used to fetch T.

For example:

- Suppose we have a relation:

  Student(RollNo, Name, Age, Department)

- To find students whose age is greater than 20:

$\{ T | T \in Student \land T.Age > 20 \}$

Here:

T is a tuple variable.

$T \in Student$ means T is a tuple of the Student relation.

T.Age > 20 is the condition.

2. Domain Relational Calculus (DRC):

- Domain Relational Calculus (DRC) is a non-procedural (declarative) query language used in relational databases.
- DRC specifies what data is required, rather than how to retrieve the data.
- Unlike Tuple Relational Calculus (TRC), which uses tuple variables, DRC uses domain variables.
- A domain variable represents the individual values (attributes) of a tuple.

Notation:

A query in DRC is written as:

{ <x1, x2, ..., xn> | P(x1, x2, ..., xn) }

Where: x1, x2, ..., xn = domain variables representing attribute values.

P(x1, x2, ..., xn): predicate/condition that specifies which values should be selected.

The values satisfying the condition form the final result.

For example:

- **Problem:** Find the account-number, customer-name, and balance for accounts with a balance less than $5000.
- **Relation:** Account(account_no, cust_name, balance)
- { <acc, c, b> | <acc, c, b> ∈ Account ∧ b < 5000 }
- **Explanation:**
- <acc, c, b> are domain variables representing account number, customer name, and balance.
- ∈ Account: the tuple must be in the Account relation.
- b < 5000: only include accounts with balance less than 5000.

VIEW:

- “views” are virtual relation(or virtual tables), through which a selective portion of the data from one or more relations (or tables) can be seen.
- Views do not contain data of their own.
- View do not exist physically.

Use of view:

- ✓ It help in query processing like simplify commands for the user, store complex queries, etc.
- ✓ To hide data complexity.
- Database modifications (like insert, delete and update operations) on views affects the actual relations in the database, upon which view is based.
- View are stored in the data dictionary in the table called USER_VIEWS

- A view is defined using the create view statement:

Create view v as <query expression>

Where <**query expression**> is any legal relational algebra query expression, and v represents the view name.

Syntax

CREATE VIEW view_name

ASSELECT column1, column2, ...

FROM table_name

WHERE condition;

Example

Suppose we have:

- **Student(RollNo, Name, Age, Department)**
- To create a view containing only students from the **CS department**:

```
CREATE VIEW CS_Students AS
SELECT RollNo, Name, Age
FROM Student
WHERE Department = 'CS';
```

Now we can access the view using:

```
SELECT * FROM CS_Students;
```

Basis	**TRC (Tuple Relational Calculus)**	**DRC (Domain Relational Calculus)**
Full form	Tuple Relational Calculus	Domain Relational Calculus
Variables	Uses **tuple variables**	Uses **domain variables**
Variable represents	An **entire tuple (row)**	An **individual attribute value**
Works with	Rows/tuples	Attribute values/domains
Query focus	Selects **tuples** that satisfy a condition	Selects **attribute values** that satisfy a condition
Basic notation	{ t | P(t) }	{ $<x_1, x_2, ..., x_n>$ | $P(x_1, x_2, ..., x_n)$ }
Example variable	t represents a complete row	x, y, z represent individual values
Similarity	Both are **non-procedural/declarative** query languages	Both are **non-procedural/declarative** query languages